

大语言模型及应用

Instructor: Xiaohan Ma(马晓寒), Ph.D.

Graduate School of Information and Communication Technology,

Ajou University, Korea

maxiaohan@ajou.ac.kr

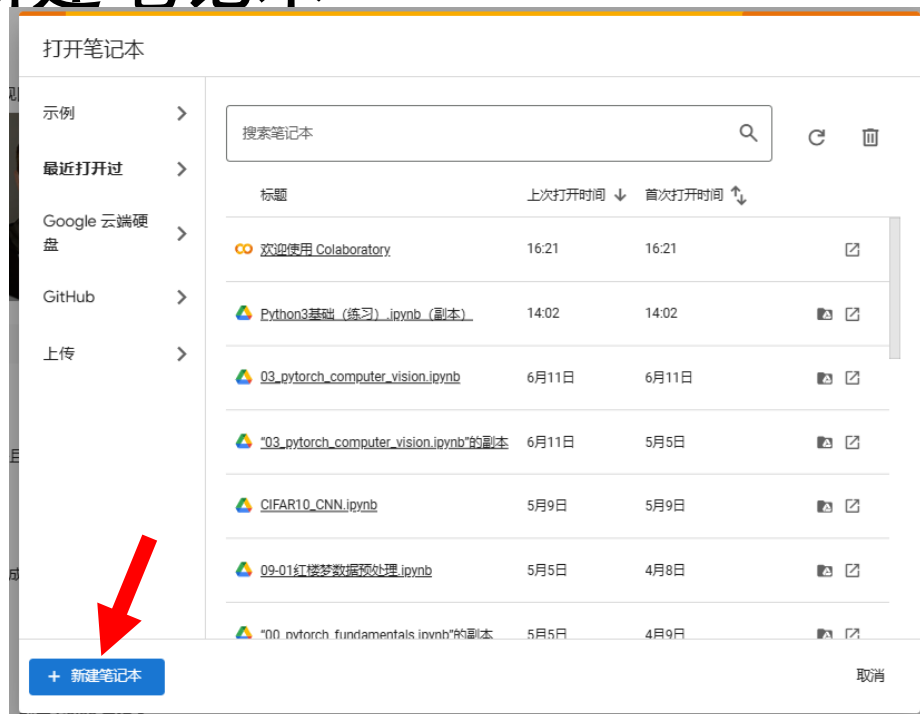
课前准备

课程准备

- **Colab (小白推荐):**
 - [欢迎使用 Colaboratory - Colaboratory \(google.com\)](#) 谷歌账户登录
 - 使用免费版即可
 - [Colab使用教程（超级详细版）及Colab Pro/Pro+评测 - 温柔的玉米 - 博客园 \(cnblogs.com\)](#)
- **本地安装（有python基础）**
 - 安装教程参考文件“本地 Conda 环境配置完整教程.docx”

Colab的使用

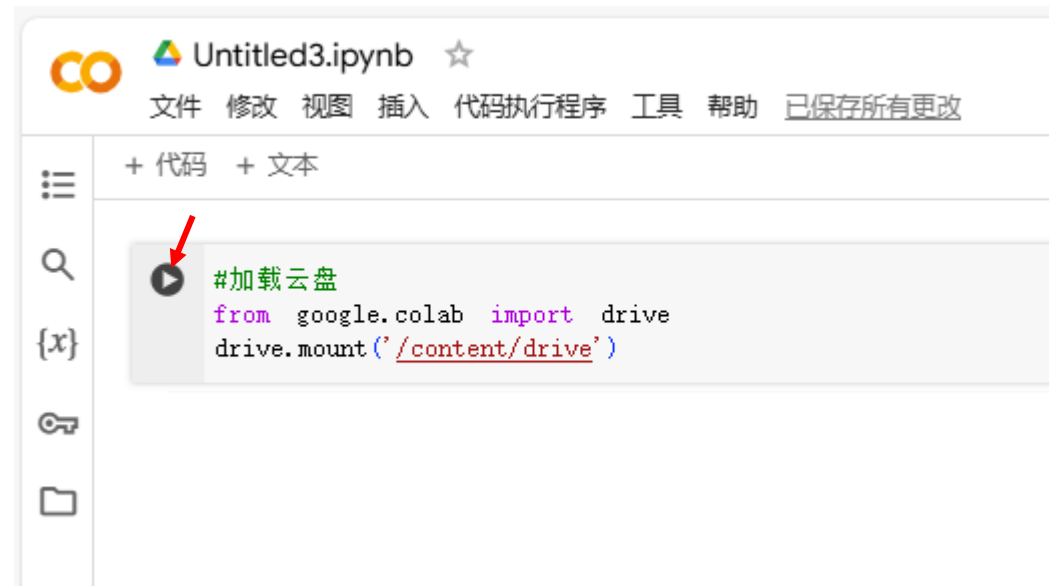
- 登陆谷歌账户，打开[欢迎使用 Colaboratory - Colab \(google.com\)](https://colab.google)
- 选择新建笔记本



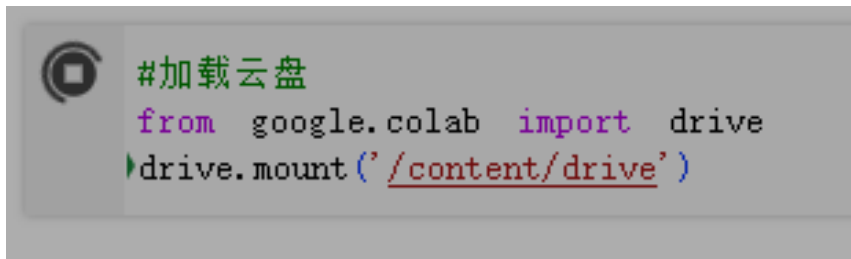


执行代码块

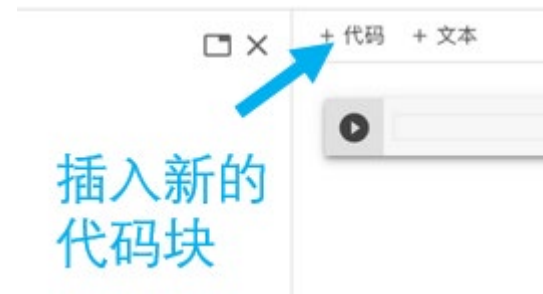
- 点击“播放”按钮执行代码块。



- 代码块开始执行后，按钮就会进入**转圈**的状态，表示“正在执行”，外部的圆圈是实线。
- 点击“+代码”可以增加新的代码块



```
#加载云盘
from google.colab import drive
drive.mount('/content/drive')
```



- 如果有代码块执行的情况下继续点击其他代码块的“播放”按钮，则这些代码块进入“等待执行”的状态，按钮也就会进入转圈的状态，但外部的圆圈是虚线。
- 在当前代码块结束后，会之前按照点击的顺序依次执行这些代码块。

Mount google drive and unzip the dataset.

```
from google.colab import drive
drive.mount('/content/drive')

!rm -r datasets
!mkdir /content/datasets
!tar -xvf "/content/drive/MyDrive/mldl_2022/ROD-synROD.tar" -C "/content/datasets" -o ouput
```

正在执行

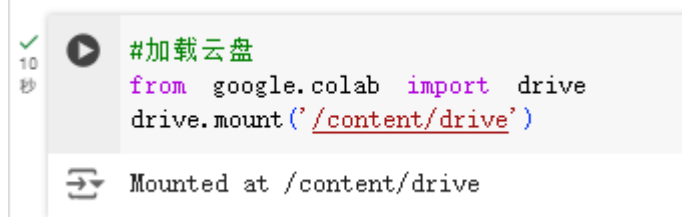
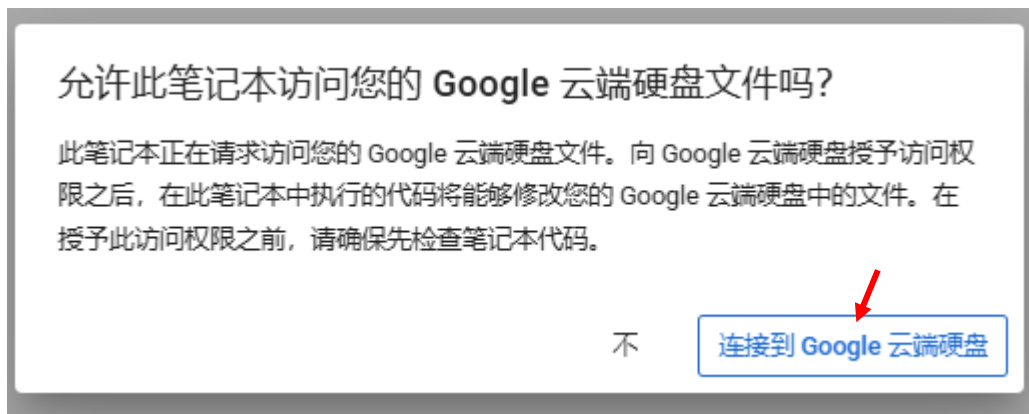
Check GPU info.

```
gpu_info = !nvidia-smi
gpu_info = '\n'.join(gpu_info)
if gpu_info.find('failed') >= 0:
    print('Not connected to a GPU')
else:
    print(gpu_info)
```

等待执行

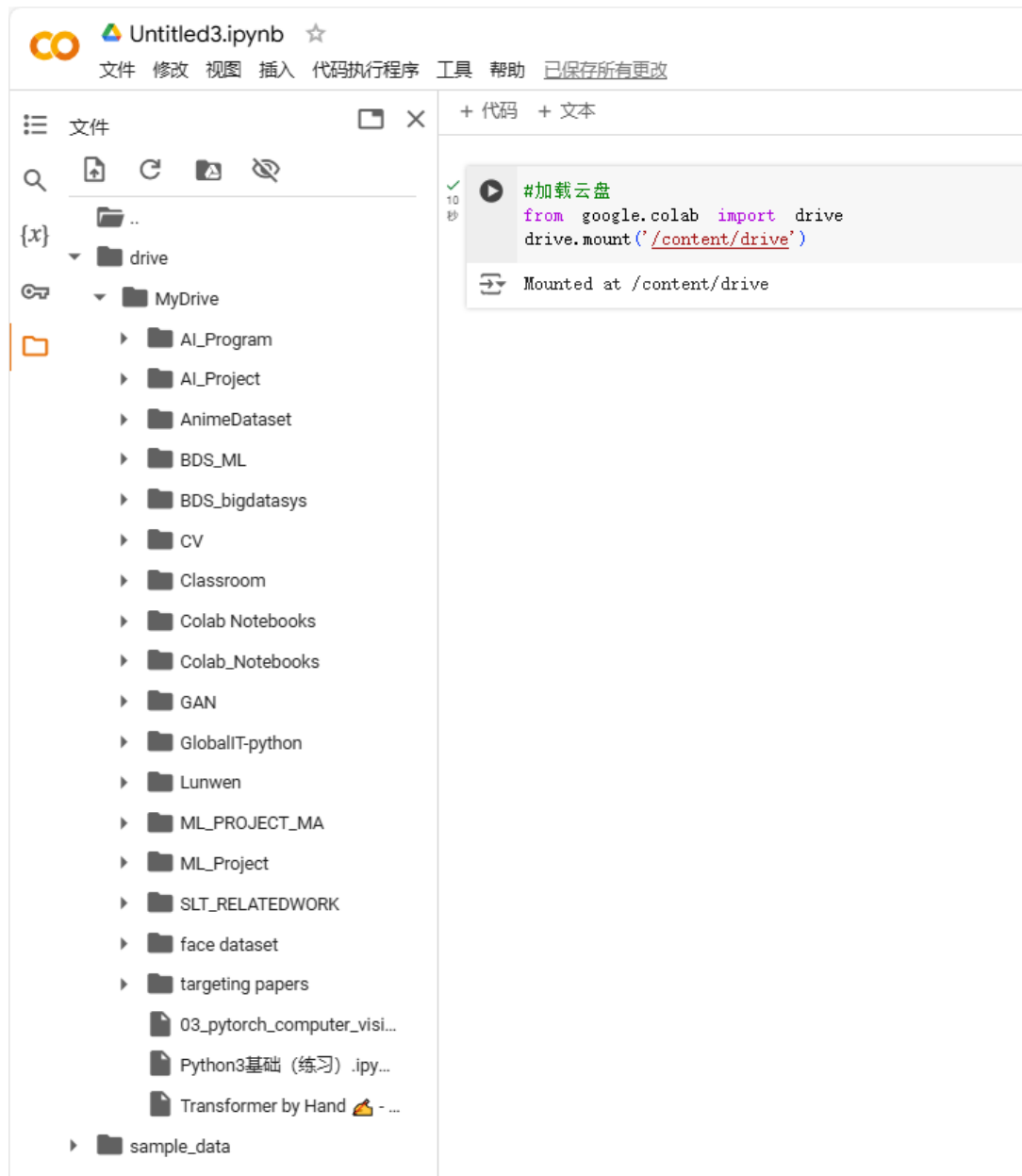
连接到谷歌硬盘

- 运行上述代码后，登录谷歌账号连接谷歌云端硬盘
- 出现右图的结果表示运行成功



连接到谷歌硬盘

- 点击左侧“文件夹”按钮，即可查看



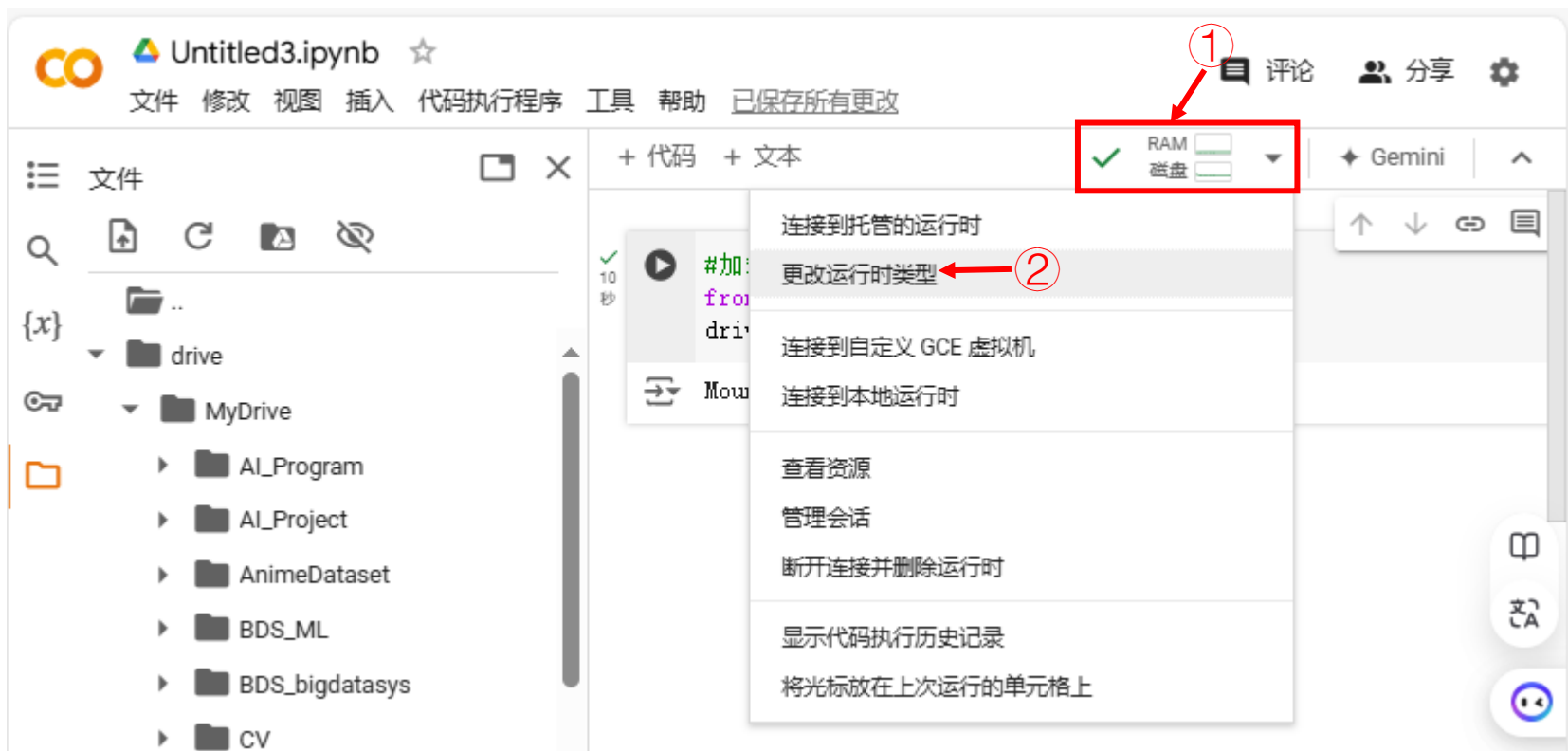
The screenshot shows a Google Colab notebook titled "Untitled3.ipynb". The left sidebar displays a file explorer with a tree structure. Under the "drive" folder, there is a "MyDrive" folder, which is expanded to show a list of subfolders and files. The right pane shows a code cell with the following Python code:

```
#加载云盘
from google.colab import drive
drive.mount('/content/drive')
```

Below the code cell, a message indicates the successful mounting of the drive:

```
Mounted at /content/drive
```

在Colab中使用GPU



在Colab中使用GPU

- 在代码块里输入“!nvidia-smi”可以查看gpu

```
[1] !nvidia-smi
```

Wed Aug 21 07:59:53 2024

NVIDIA-SMI 535.104.05			Driver Version: 535.104.05			CUDA Version: 12.2		
GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile Uncorr. ECC			
Fan	Temp	Perf	Pwr:Usage/Cap	Memory-Usage	GPU-Util	Compute M.	MIG	M.
0	Tesla T4	Off	00000000:00:04.0	Off	0			
N/A	41C	P8	9W / 70W	0MiB / 15360MiB	0%	Default		N/A

Processes:

GPU	GI	CI	PID	Type	Process name	GPU Memory Usage
ID	ID	ID				
No running processes found						

当前使用

总显卡内存

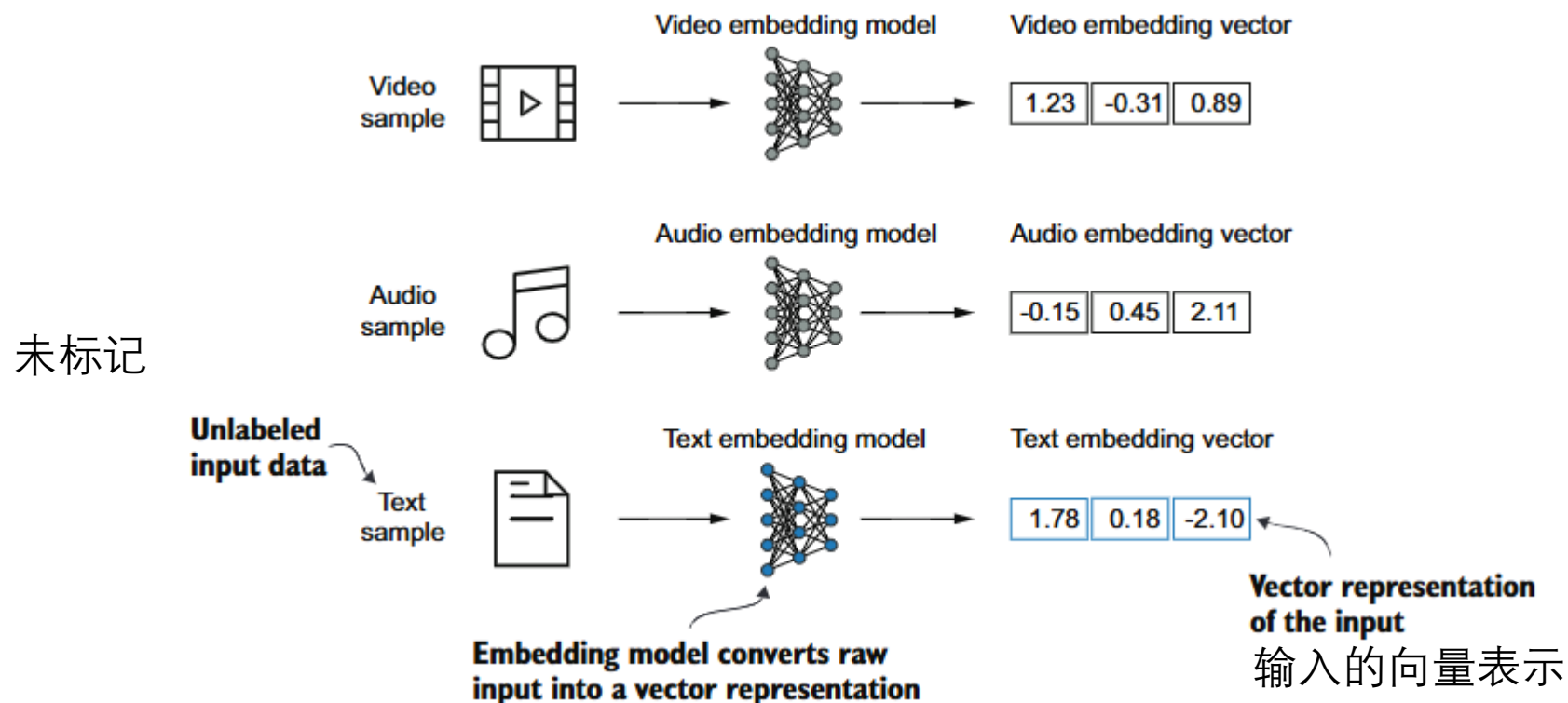
词表示问题

大纲

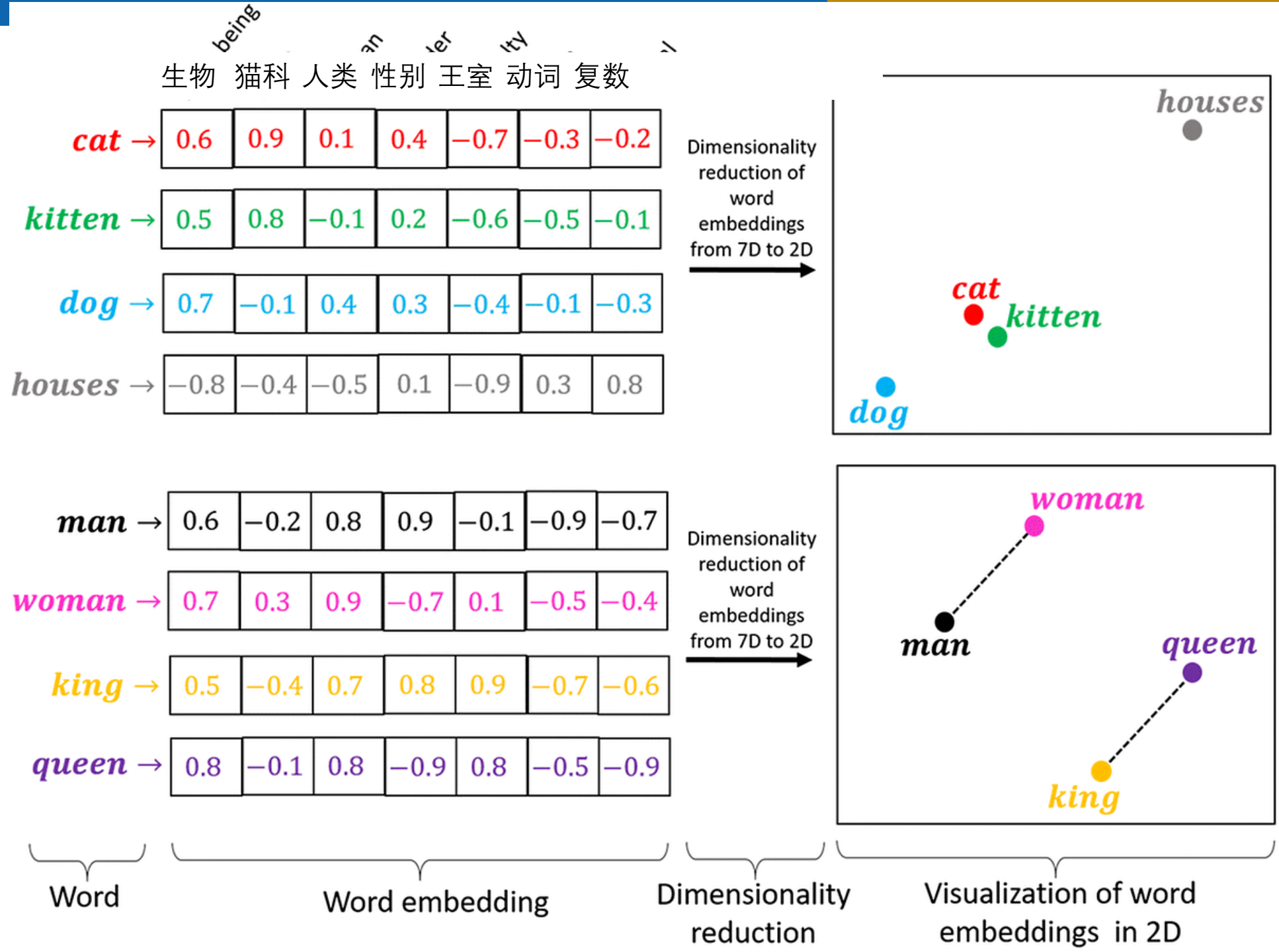
- 2.1 理解词向量
- 2.2 文本分词
- 2.3 将token 转化为token ID
- 2.4 添加特殊上下文token
- 2.5 Byte pair encoding (字节对编码)
- 2.6 使用滑动窗口进行数据采样
- 2.7 创建 token 向量
- 2.8 编码单词位置
- 本章总结

2.1 理解词向量(word embeddings)

- embedding是从离散对象（例如单词、图像甚至整个文档）到连续向量空间中的点的映射 - 向量的主要目的是将非数字数据转换为神经网络可以处理的格式。



词向量模型将原始输入转换为向量表示



2.1 理解词向量(word embeddings)

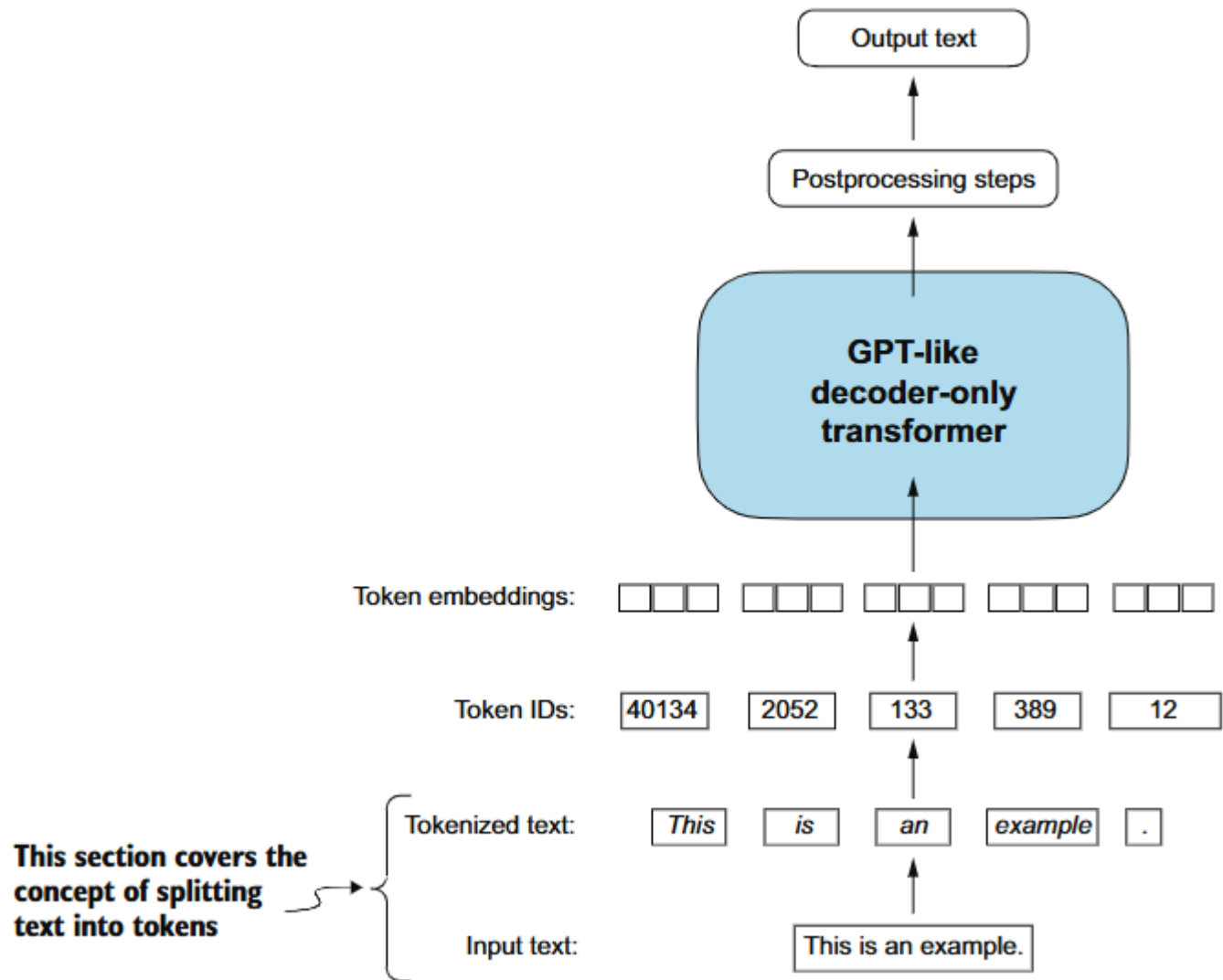
- 在使用 LLMs 时，我们通常使用高维度的向量。

模型	参数量	向量大小（隐藏状态维度）	特点
GPT-2	117M / 125M	768维	计算需求低，适用于轻量级 NLP 任务
GPT-3	175B	12,288 维	语义理解更强，计算成本更高，适用于复杂任务

向量大小（Embedding Size）决定了模型的计算复杂度和能力，需要在 性能 vs. 计算效率 之间做出权衡！

2.2 文本分词

- 将输入文本拆分为单个 **token**，这是创建向量的必需预处理步骤



What is Tokenization (分词)?

分词是将文本分割成若干片段（称为词元）的任务

Token（词元）是某个具体文档中字符序列的一次出现实例（通常被当作一个有意义的语义单位）。

Type（词型）是所有具有相同字符序列的 **Token** 的类别。

Term（词项）是一种（可能经过规范化处理的）Type，并且被包含在“语料库词典（corpus dictionary）”中。

Example: to sleep more to learn

Token: to, sleep, more, to, learn

Type: to, sleep, more, learn

Term: sleep, more, learn (stop words removed)

分词问题

- **Maine's state capital**
 - Maine, Maines, Maine's?
- **Dover-Foxcroft**
 - 是1个token, 还是2个token: Dover, Foxcroft?
 - State-of-the-art: 是否应该把带连字符的词拆开?
- **San Francisco**: 1个 token 还是2个?
- **数字的处理**
 - September 13, 2022
 - 13/9/2022
 - (686)753-2910
 - 一些较早的信息检索系统 (IR systems) 可能不会对数字建立索引
- 系统通常会对元数据单独建立索引



Dover-Foxcroft, Maine

分词问题(其他语言)

- **French 法语**
 - L'ensemble → 是1个token 还是2个?
 - 应该拆分成 L ? L' ? Le ?
 - 希望 l'ensemble 可以与 un ensemble 匹配
- **German 德语**
 - 德语中的复合名词通常不会被分割
Kraftfahrzeughaftpflichtversicherung 含义: 机动车责任保险
- **Chinese & Japanese 中文&日语**
 - 词之间没有空格
 - 自然语言处理课程 → 'Natural Language Processing course'
- **Persian & Arabic 波斯语和阿拉伯语**
 - 从右向左书写
 - Letters can be connected together
 - کلاس طبیعی ابوشیراز اوپس لک
 - 'Natural language processing class'

基于 word 的分词

方法

- 通过空格 (space) 将文本拆分成词
- 也可以使用其他分隔符, 例如标点符号

优点

- 实现简单

缺点

High risk of missing words; e.g., Let and Let's will have two different types

- 容易遗漏词形变化例如: Let 和 Let's 会被当作两个不同的词类型
- 某些语言没有空格例如: 中文
- 词汇表规模会非常大 (token type 数量巨大)
- 需要限制词汇表的大小
- 拼写错误的单词也会被当作新的 token

基于字符的分词

方法

- 将文本拆分为单个字符

优点

- 几乎不会出现未知词 (Out Of Vocabulary, OOV)
- 对于字符本身具有语义信息的语言 (如中文) 比较有用
- 词表规模较小 (字符数量有限)
- 实现简单

缺点

- 单个字符通常没有完整的语义
- 难以学习词级别的语义信息
- 需要处理的序列会更长
- 模型需要处理更多输入, 计算成本更高

what is glablahglee?

I'm sorry, but I couldn't find any information about "glablahglee". Could you provide more context or clarify the term you're asking about?

子词分词

方法

- 常见词 (frequent words) 不应被拆分为更小的子词
- 罕见词 (rare words) 应被拆分为有意义的子词
- 使用特殊符号来表示: 一个 token 是否是词的开头或者是词的一部分
 - Tokenization → “Token”, “ization” 其中: Token 表示词的开头, ization 表示该子词是前面词的一部分
- 目前许多自然语言处理 (NLP) 和 信息检索 (IR) 的最先进方法 (state-of-the-art approaches) 都依赖这种分词方式。

优点

- 解决了未知词 (Out-of-Vocabulary, OOV) 问题
- 词汇表规模更加可控 (manageable vocabulary size)

缺点 (Disadvantages)

是一种较新的方法, 仍然需要进一步研究和探索

BPE (Byte Pair Encoding) 和 **WordPiece** 是这种分词方法的两个典型例子。

字节对编码 (Byte-Pair Encoding, BPE) 分词

BPE 是一种用于分词的算法，其思想类似于 Huffman 编码，采用贪心算法 (greedy algorithm)。

训练步骤 (Training Steps)

1. 将输入单词拆分为单个字符，每个字符都对应最终词表中的一个符号。
2. 在实际应用中通常会在单词结尾添加一个特殊符号 `</w>`，用于表示单词结束。
3. 统计当前词表中出现频率最高的符号对 (pair of symbols) 将该符号对合并为一个新的 token 并将其加入词表，此时词表大小增加 1。
4. 重复步骤 2 和步骤 3，直到：构建出预先设定数量的 token，或不再存在满足频率要求的新符号组合。

Byte-Pair Encoding (BPE) Tokenization

Training corpus: low low low low low lowest lowest newer newer newer newer
newer newer wider wider wider new new

Corpus

5 l o w ____
2 l o w e s t ____
6 n e w e r ____
3 w i d e r ____
2 n e w ____

Vocabulary

____, d, e, i, l, n, o, r, s, t, w



Byte-Pair Encoding (BPE) Tokenization

Training corpus: low low low low low lowest lowest newer newer newer newer
newer newer wider wider wider new new

Corpus

5 l o w ____
2 l o w e s t ____
6 n e w e r ____
3 w i d e r ____
2 n e w ____

9 times

Vocabulary

____, d, e, i, l, n, o, r, s, t, w

Vocabulary

____, d, e, i, l, n, o, r, s, t, w, er

Byte-Pair Encoding (BPE) Tokenization

Training corpus: low low low low low lowest lowest newer newer newer newer
newer newer wider wider wider new new

Corpus

9 times

5 l o w ____
2 l o w e s t ____
6 n e w e r ____
3 w i d e r ____
2 n e w ____

Vocabulary

____, d, e, i, l, n, o, r, s, t, w, er

Vocabulary

____, d, e, i, l, n, o, r, s, t, w, er, er____

Byte-Pair Encoding (BPE) Tokenization



Training corpus: low low low low low lowest lowest newer newer newer newer
newer newer wider wider wider new new

	Corpus	Vocabulary
8 times	5 low ____	____, d, e, i, l, n, o, r, s, t, w, er, er____
	2 low est ____	
	6 new er ____	Vocabulary
	3 wider ____	
	2 new ____	____, d, e, i, l, n, o, r, s, t, w, er, er____, ne

Byte-Pair Encoding (BPE) Tokenization

Training corpus: low low low low low lowest lowest newer newer newer newer
newer newer wider wider wider new new

Corpus

8 times

5 l o w ____
2 l o w e s t ____
6 n e w e r ____
3 w i d e r ____
2 n e w ____

Vocabulary

____, d, e, i, l, n, o, r, s, t, w, er, er____, ne

Vocabulary

____, d, e, i, l, n, o, r, s, t, w, er, er____, ne, new

Byte-Pair Encoding (BPE) Tokenization



语料库

5 l o w _
2 l o w e s t ____
6 n e w e r ____
3 w i d e r .
2 n e w _

Final Vocabulary

____, d, e, i, l, n, o, r, s, t, w, er, er____, ne, new, lo, low, newer____, low____

根据我们学到的顺序合并:

er → er____ → ne → new → newer____

er → er____ → lo → low

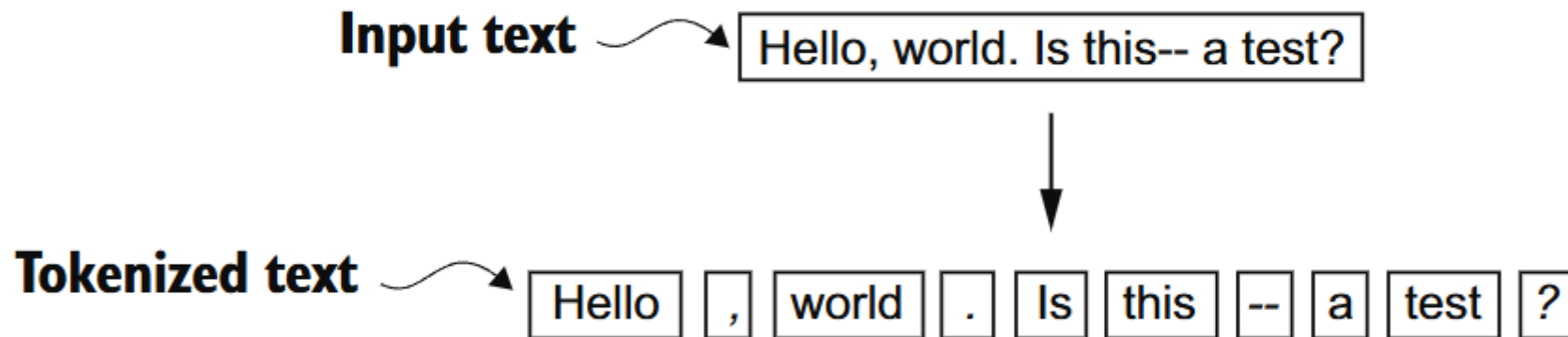
使用BPE进行分词:

Input: newer____ → Tokens: newer____

Input: lower____ → Tokens: low, er____

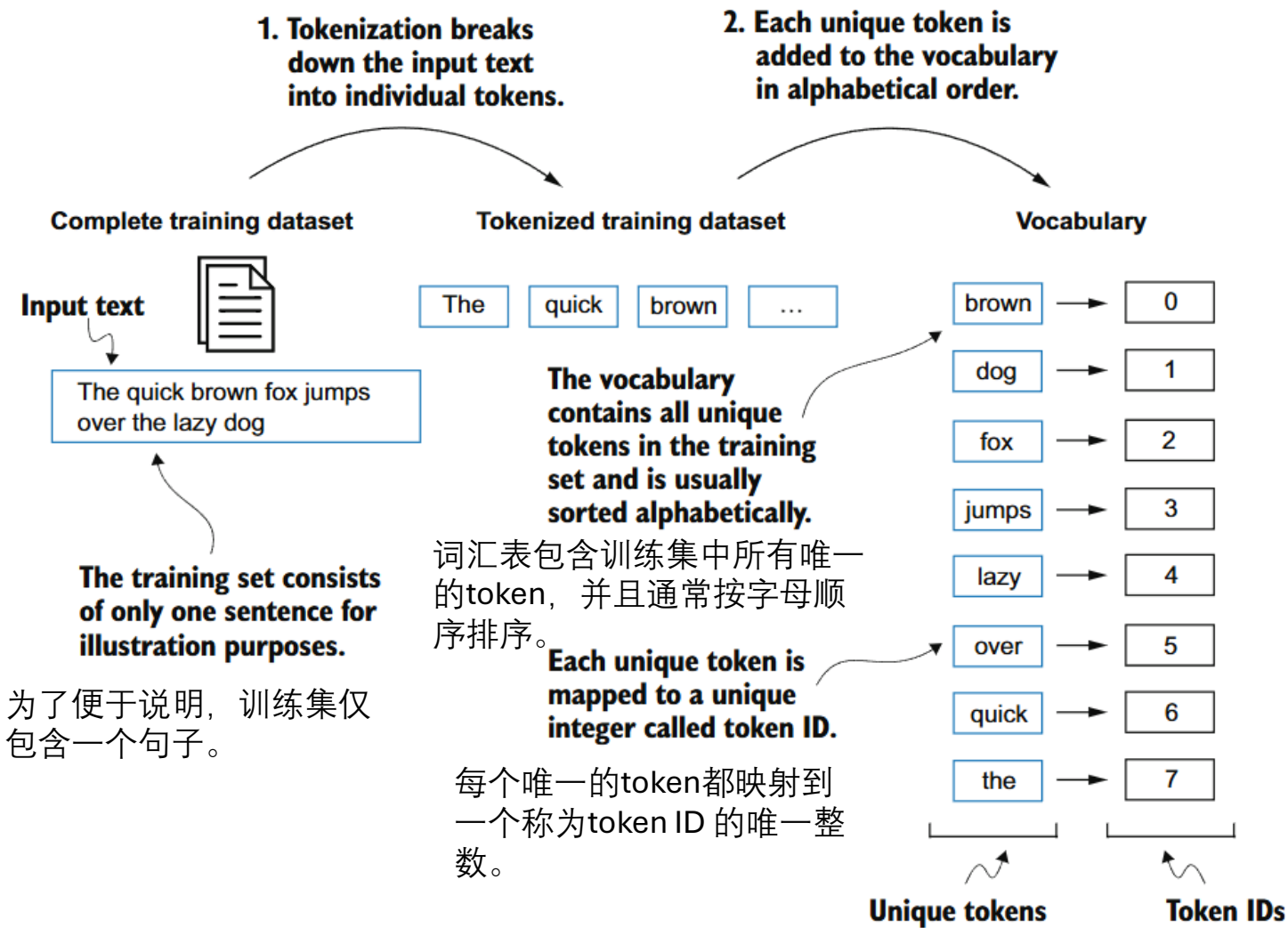
如何拆分文本? (演示)

- 增加分割符，以便处理其他类型的标点符号，例如问号、引号，破折号。
- 以文本"Hello, world. Is this-- a test?"为例



2.3 将token转换为token ID

1. 标记化将输入文本分解为单个token。
2. 每个唯一的token按字母顺序添加到词汇表中。

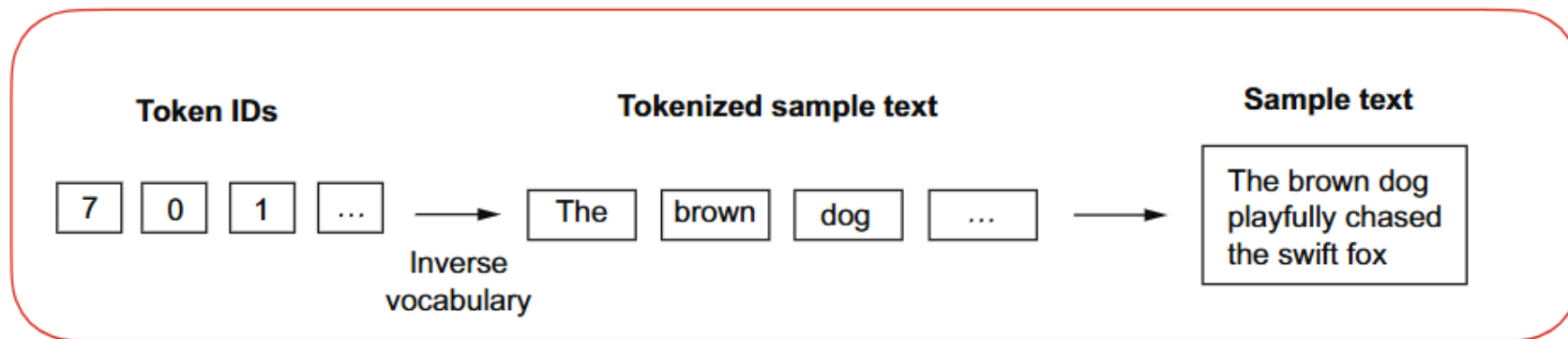
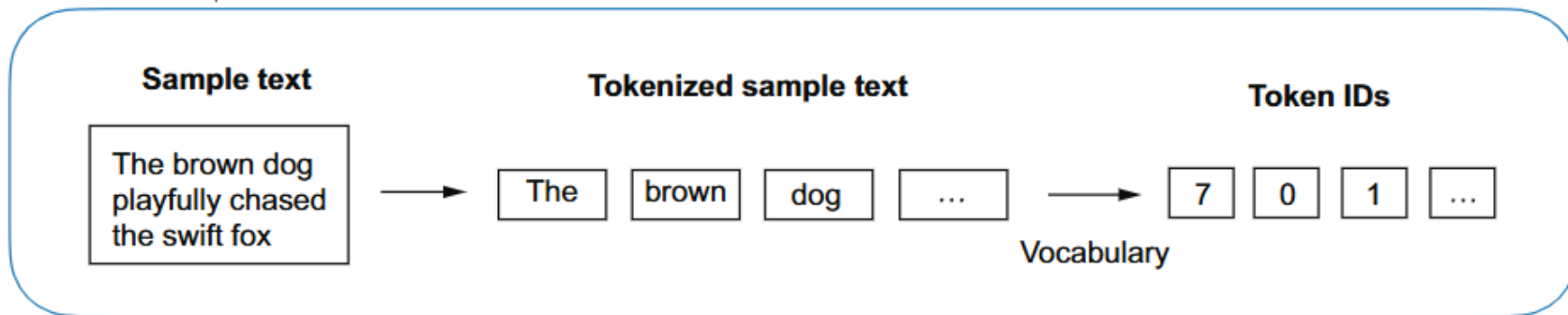


2.3 将token转换为token ID

- 一个简单的文本分词器 (Tokenizer) , 核心功能:
 - 存储词汇表 (vocab)
 - 包含一个 **encode** 方法, 该方法将文本拆分为 **token**, 并执行字符串到整数的映射, 以通过词汇表生成 **token ID**。
 - 一个 **decode** 方法, 该方法执行反向整数到字符串的映射, 以将 **token ID** 转换回文本。

2.3 将token转换为token ID

Calling `tokenizer.encode(text)` on sample text



Calling `tokenizer.decode(ids)` on token IDs

2.4 添加特殊上下文token

Sample text

The brown dog
playfully chased
the swift fox

Tokenized sample text

The brown dog playfully ...

Existing vocabulary

brown	→	0
dog	→	1
fox	→	2
...	→	...

Extend vocabulary
with additional
special tokens

< unk >	→	783
< endoftext >	→	784

Token IDs

7 0 1 783 ...

用附加特殊token来扩展词汇量

2.4 添加特殊上下文token

- 一些分词器 (tokenizer) 使用特殊token (special tokens) 来帮助 LLM 提供额外的上下文信息。
 - [BOS] (beginning of sequence) : 表示文本的起始。
 - [EOS] (end of sequence)
表示文本的结束 (通常用于连接多个不相关的文本, 例如两篇不同的维基百科文章或两本不同的书籍)。
 - [PAD] (padding) : 当 LLM 以批量 (batch size > 1) 训练时, 不同的文本长度可能不同。通过 PAD 填充较短的文本, 使所有文本长度相同, 以便批量处理。
 - [UNK] (unknown) : 用于表示不在词汇表中的单词。

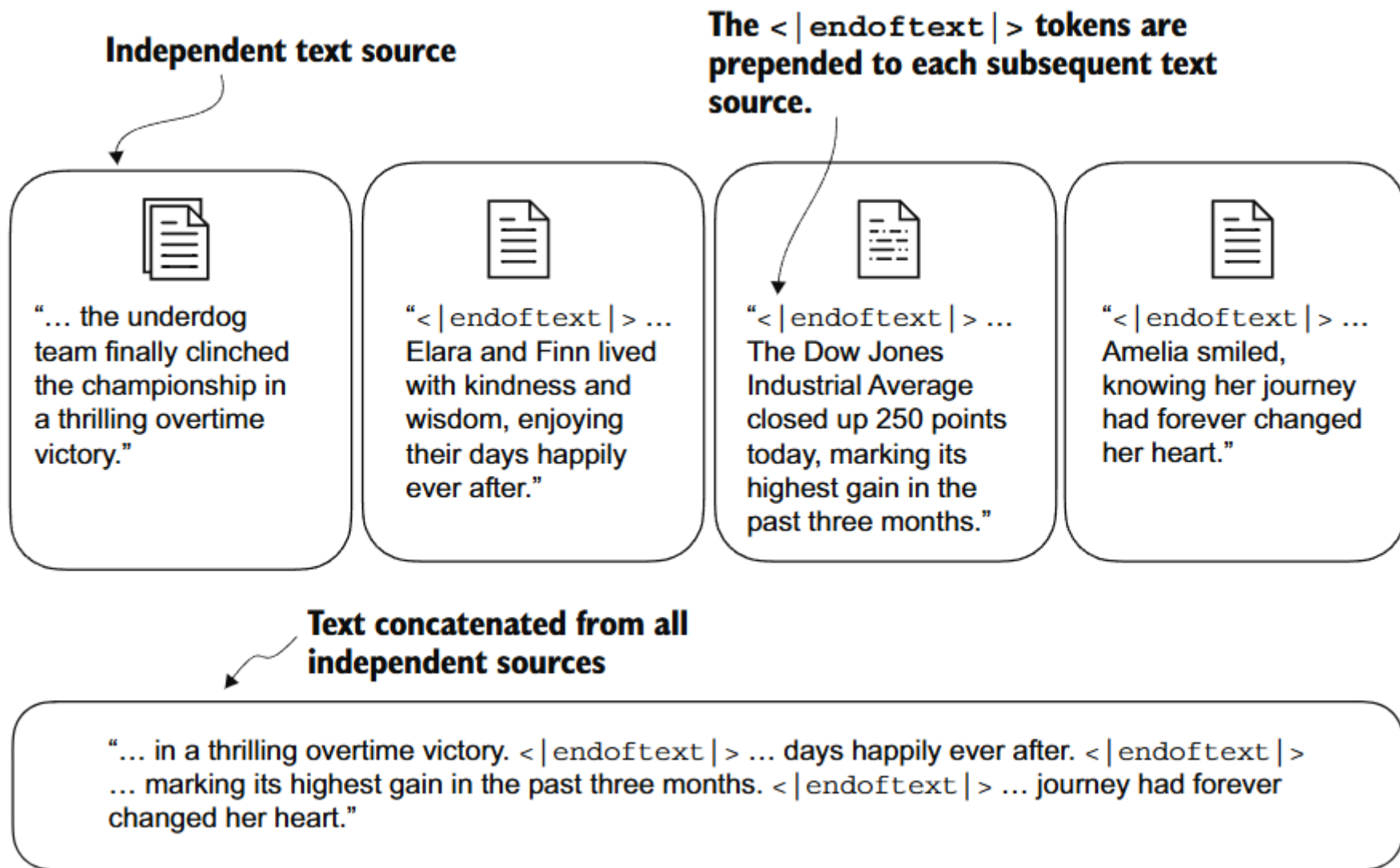
2.4 添加特殊上下文token

- 不属于词汇表的单词，可以修改标记器以使用 “<|unk|>” 标记
- 在多个独立文档或书籍上进行类似 GPT 的训练时，通常会在每个文档或书籍之前插入一个标记 “<|endoftext|>”，这些文档或书籍紧跟前一个文本源



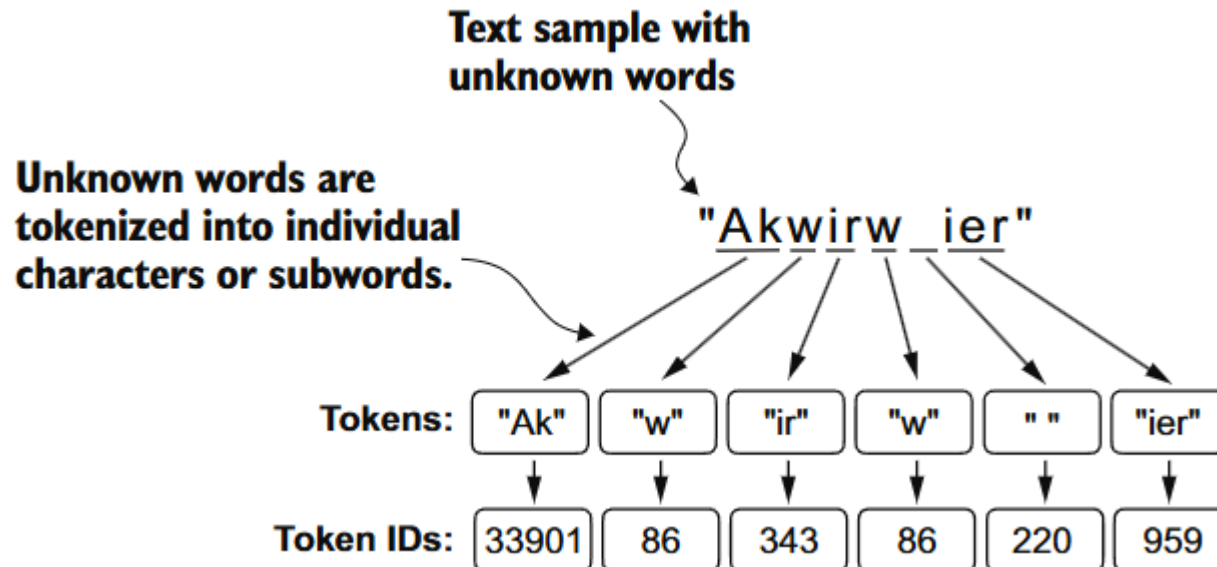
2.4 添加特殊上下文标记

当处理多个独立文本源时，我们在这些文本之间添加 `<|endoftext|>` 标记。`<|endoftext|>` 标记充当标记，表示特定片段的开始或结束，以便更有效地处理和被LLM理解。



2.5 Byte pair encoding (BPE 字节对编码)

- GPT-2 使用 BPE 作为分词器 (Tokenizer) 。
- BPE 可以将 OOV (词汇表外单词) 拆分成更小的子单元 (subword units)，甚至可以拆分为单个字符。
- 提高词汇表的覆盖范围，减少未知词 (OOV) 问题。



2.5 Byte pair encoding (BPE 字节对编码)

- GPT-2 的 BPE 实现

- GPT-2 最初的 BPE 分词器可以在 OpenAI 官方仓库<https://github.com/openai/gpt-2/blob/master/src/encoder.py> 找到 OpenAI 的 tiktoken 分词器
- 支持 GPT-2、GPT-3、GPT-4 的 BPE 分词。适用于高效的 Token 处理，在推理 (inference) 和训练 (training) 时减少计算开销。

- 如何使用 BPE?

- 使用 `pip install tiktoken` 安装 tiktoken 库

✓
14
秒

```
[1] !pip install tiktoken
```



```
Collecting tiktoken
  Downloading tiktoken-0.9.0-cp311-cp311-
Requirement already satisfied: regex>=202
Requirement already satisfied: requests>=
Requirement already satisfied: charset-no
Requirement already satisfied: idna<4,>=2.
Requirement already satisfied: urllib3<3,
Requirement already satisfied: certifi>=2
Downloading tiktoken-0.9.0-cp311-cp311-mar
-----
Installing collected packages: tiktoken
Successfully installed tiktoken-0.9.0
```

2.5 Byte pair encoding (BPE 字节对编码)

- 如何使用BPE?
 - 加载 OpenAI GPT-2 预训练模型的 BPE 分词器。返回 Encoding 对象，可以用于文本的 tokenization（分词）和 detokenization（还原文本）。

```
▶ import importlib  
import tiktoken  
importlib.metadata.version("tiktoken")
```

```
... '0.12.0'
```

2.5 Byte pair encoding (BPE 字节对编码)

- 如何使用BPE?
 - BPE分词器类型

分词器	词汇表大小	主要支持语言
GPT-2 (gpt2)	50,257	主要是 英语
GPT-3 (p50k_base)	50,257	英语、法语、西班牙语、部分其他语言
GPT-3.5/GPT-4 (cl 100k_base)	100,000+	英语、法语、西班牙语、中文、日语、韩语、阿拉伯语等多种语言

2.5 Byte pair encoding (BPE 字节对编码)

- 如何使用BPE?
 - 选用GPT-2的分词器

✓
1 秒



```
tokenizer = tiktoken.get_encoding("gpt2")  
tokenizer
```



<Encoding 'gpt2' >

2.5 Byte pair encoding (BPE 字节对编码)

- 如何使用BPE?
 - 示例-进行文本编码 (Tokenization)

```
[58] text = (  
    "Hello, do you like tea? <|endoftext|> In the sunlit terraces"  
    "of someunknownPlace." )  
  
integers = tokenizer.encode(text, allowed_special={"<|endoftext|>"})  
  
print(integers)
```

→ [15496, 11, 466, 345, 588, 8887, 30, 220, 50256, 554, 262, 4252, 18250, 8812, 2114, 1659, 617, 34680, 27271, 13]

2.5 Byte pair encoding (BPE 字节对编码)

- 如何使用BPE?
 - 示例-解码 Token

```
strings = tokenizer.decode(integers)  
  
print(strings)
```

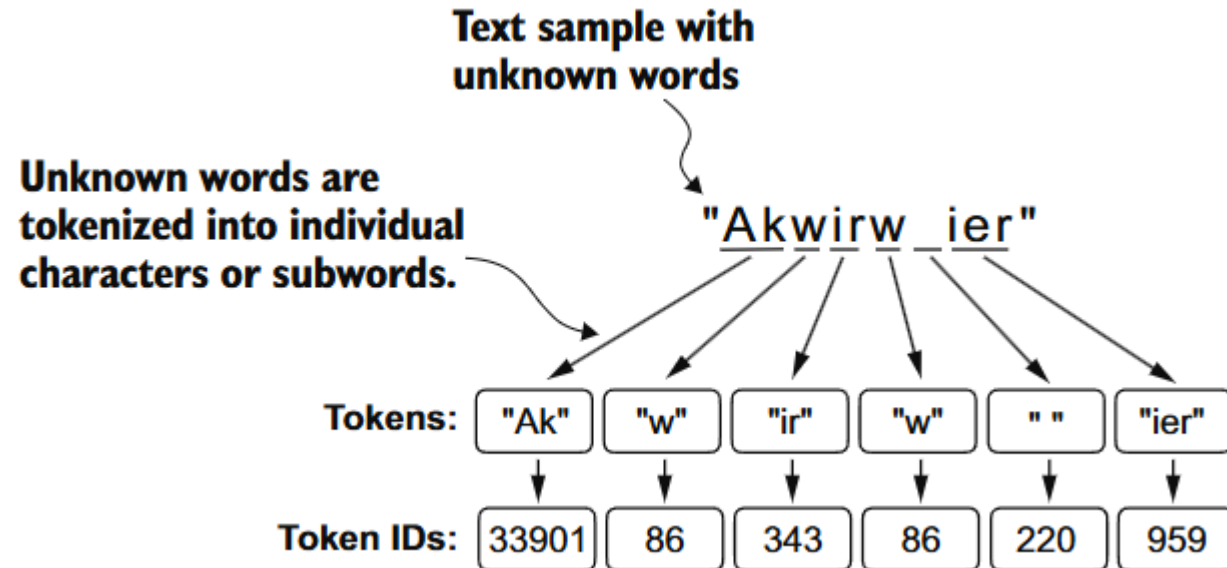
↔ Hello, do you like tea? <|endoftext|> In the sunlit terracesof someunknownPlace.

2.5 Byte pair encoding (BPE 字节对编码)

- 当BPE遇到未知词 (OOV) 时

```
[55] oov="Akwirw ier"  
     oov_id = tokenizer.encode(oov)  
  
     print(oov_id)
```

→ [33901, 86, 343, 86, 220, 959]

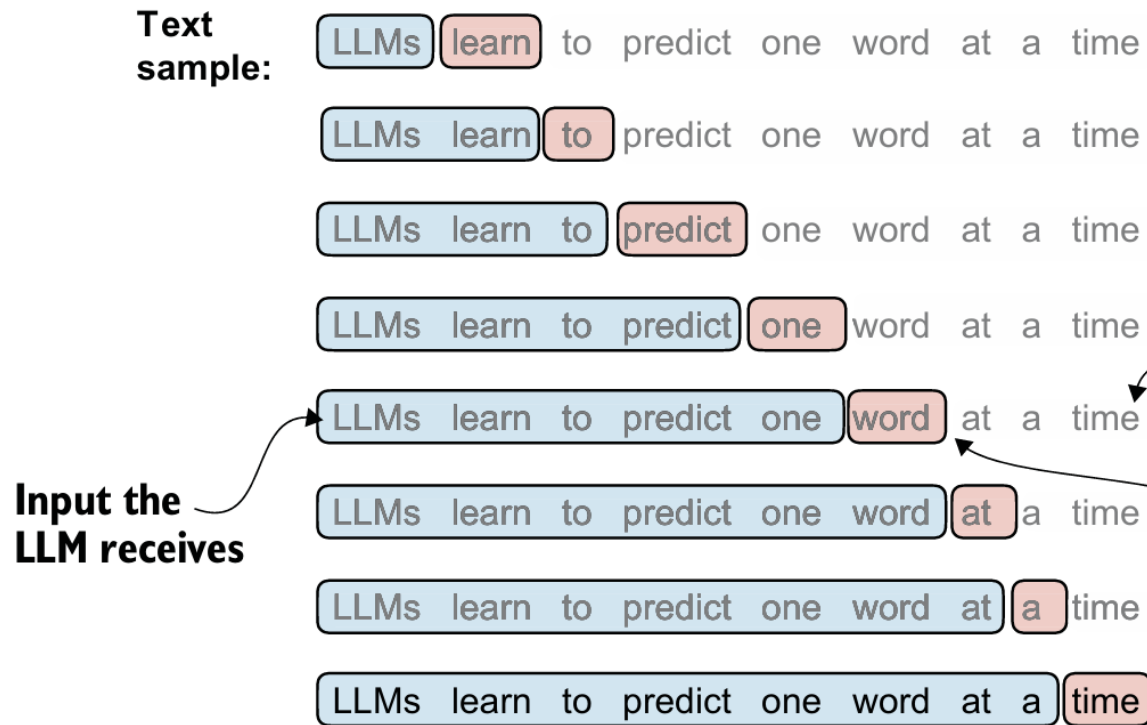


练习：使用 **BPE** 进行分词

- 请使用 Python 的 tiktoken 库，对以下中文句子进行 **BPE** 分词，并输出对应的 token 和 token id。
- 句子：text=“我正在学习Natural Language Processing，非常有意思！”
- 要求
 - 使用 tiktoken 库
 - 使用 GPT-4 / GPT-3.5 使用的 tokenizer (cl100k_base)
- 输出：
 - token id
 - token 对应的文本
- 写出最终的 分词结果

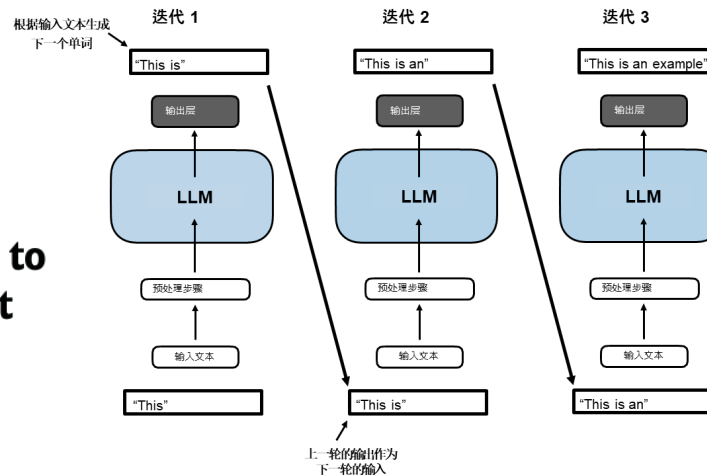
2.6 使用滑动窗口进行数据采样

- 通过预测文本中的下一个单词来预训练



The LLM can't access words past the target.

Target to predict

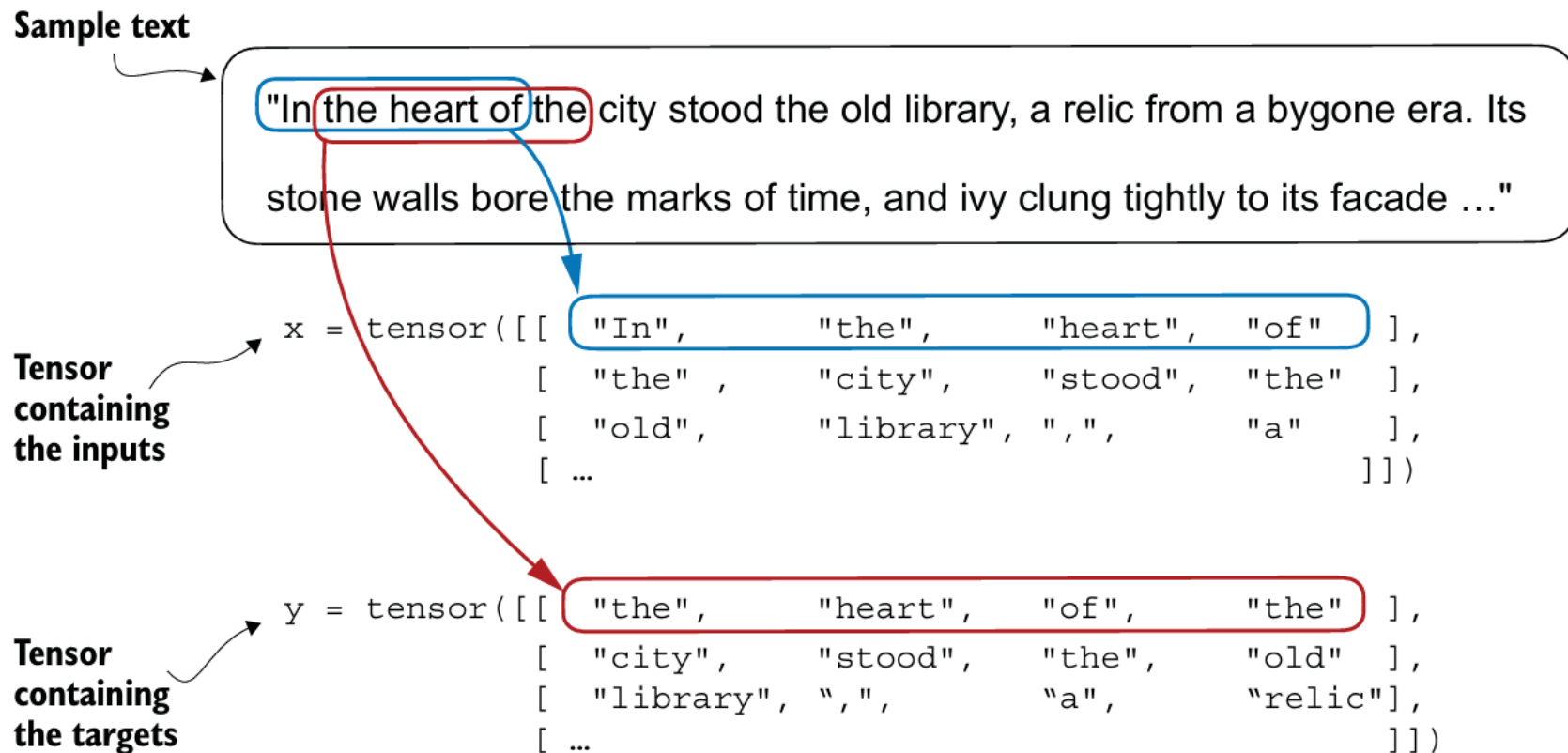


GPT训练和推理的区别

- **训练时（固定窗口，多个 token 并行预测）：**
 - 输入 x: ["In", "the", "heart", "of"]
 - 目标 y: ["the", "heart", "of", "the"]
 - 训练目标：让模型同时学会预测 "the"、"heart"、"of"、"the"
- **推理时（自回归生成，每次预测 1 个 token）：**
 - 输入 ["In"] → 预测 "the"
 - ["In", "the"] → 预测 "heart"
 - ["In", "the", "heart"] → 预测 "of"
 - ["In", "the", "heart", "of"] → 预测 "the"
 - 持续迭代，逐步生成完整文

2.6 使用滑动窗口进行数据采样

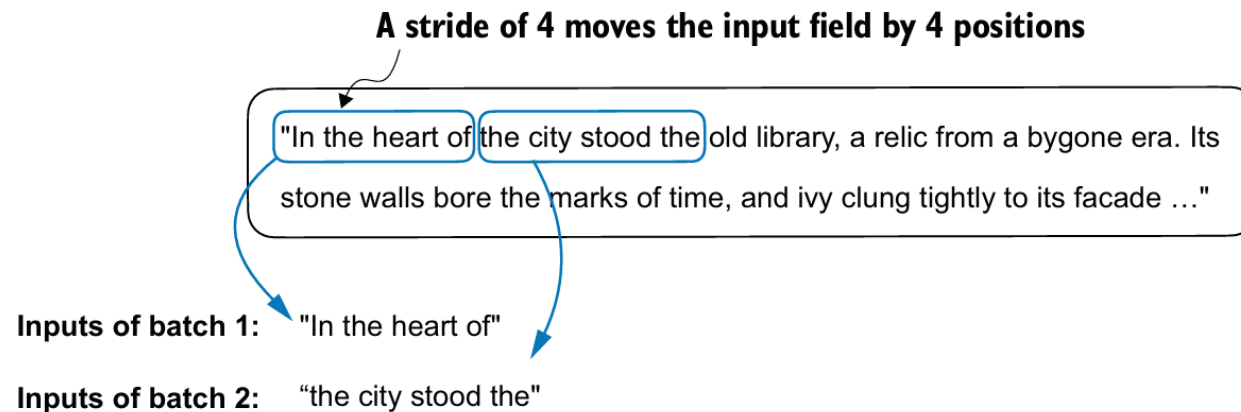
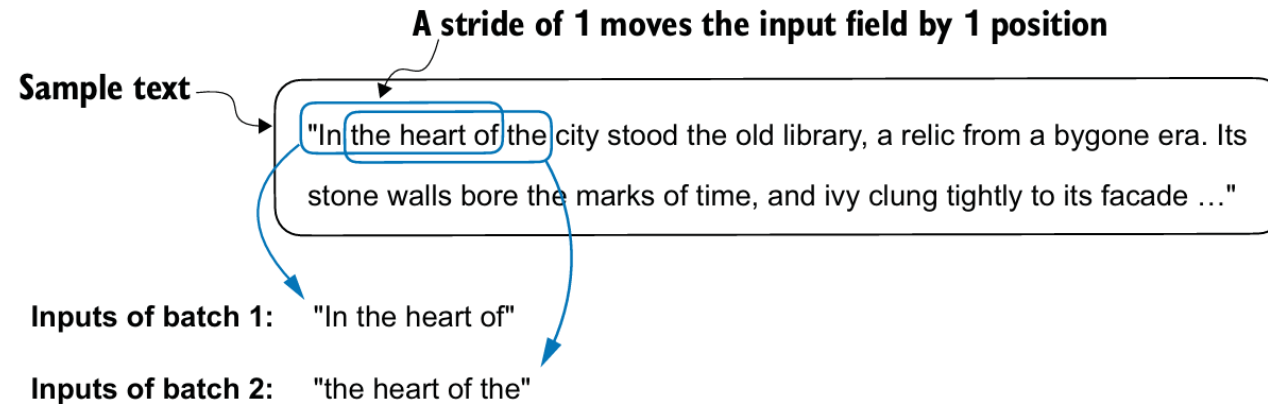
- 实现一个高效的数据加载器，该加载器迭代输入数据集并返回 PyTorch 张量形式的输入和目标张量
 - 训练时使用固定窗口 `max_length`，确保批量计算高效。



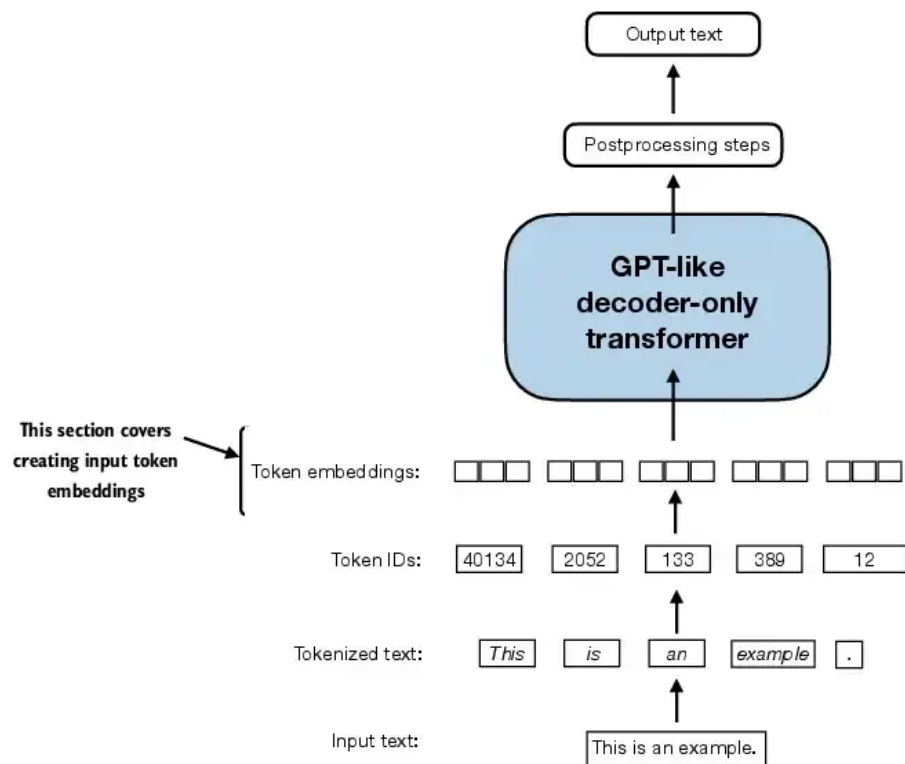
2.6 使用滑动窗口进行数据采样

- 当从输入数据集创建多个批次 (batches) 时，我们会在文本上滑动一个输入窗口 (input window) 。
- 如果步长 (stride) 设置为 1，在创建下一个批次时，输入窗口会向前移动一个位置。
- 如果将步长设置为等于输入窗口的大小，则可以避免不同批次之间出现重叠 (overlap)

2.6 使用滑动窗口进行数据采样



2.7 创建 token 向量



- 将token ID转换为token向量
 - 首先用随机值初始化，后续通过反向传播算法进行权重优化。

2.7 创建 token 向量

- 通过一个实际示例来了解 token ID 到嵌入向量转换的工作原理
 - 假设有以下四个输入 token，其 ID 分别为 2、3、5 和 1

```
▶ input_ids = torch.tensor([2, 3, 5, 1])
```

- 假设我们的词汇表只有 6 个单词（而不是 BPE 标记器词汇表中的 50,257 个单词），并且我们想要创建大小为 3 的向量（在 GPT-3 中，向量大小为 12,288 维）

```
▶ vocab_size = 6  
output_dim = 3  
  
torch.manual_seed(123)  
embedding_layer = torch.nn.Embedding(vocab_size, output_dim)
```

使用 vocab_size 和 output_dim，我们可以在 PyTorch 中实例化一个嵌入层，并将随机种子设置为 123 以实现可重复性

扩展--torch.nn.Embedding(num_idx, out_dim)

- Pytorch中的嵌入层torch.nn.Embedding(num_idx, out_dim)
 - 将离散索引（如单词ID、类别ID）映射为连续向量的层，
 - num_idx 为词汇表大小， out_dim为词向量维度。
 - 嵌入层矩阵形状为num_idx × out_dim
 - 例如 num_idx = 4， out_dim=5， 则嵌入层矩阵形状为4 × 5
 - torch.manual_seed(123)
 - embedding = torch.nn.Embedding(num_idx, out_dim)

In [4]:

```
embedding.weight
```

Out[4]: Parameter containing:

```
tensor([[ 0.3374, -0.1778, -0.3035, -0.5880,  1.5810],  
        [ 1.3010,  1.2753, -0.2010, -0.1606, -0.4015],  
        [ 0.6957, -1.8061, -1.1589,  0.3255, -0.6315],  
        [-2.8400, -0.7849, -1.4096, -0.4076,  0.7953]], requires_grad=True)
```


扩展--torch.nn.Embedding(num_idx, out_dim)

- Embedding 的作用类似于 查表 (Lookup Table) :
- 它维护一个大小为 $(\text{num_idx} \times \text{out_dim})$ 的 嵌入矩阵 W 。
- 输入 token ID i , 直接查表返回 $W[i]$ 这一行的向量 , 而不进行矩阵乘法。

1) Index of training example [1]

2) Embedding matrix

0.3374	-0.1778	-0.3035	-0.5880	1.5810
1.3010	1.2753	-0.2010	-0.1606	-0.4015
0.6957	-1.8061	-1.1589	0.3255	-0.6315
-2.8400	-0.7849	-1.4096	-0.4076	0.7953

3) Look up entries by index

[1.3010 1.2753 -0.2010 -0.1606 -0.4015]

In [5]:

```
embedding(torch.tensor([1]))
```

Out[5]:

```
tensor([[ 1.3010,  1.2753, -0.2010, -0.1606, -0.4015]],  
       grad_fn=<EmbeddingBackward0>)
```

[1]

扩展--torch.nn.Embedding(num_idx, out_dim)

```
In [6]: embedding(torch.tensor([2]))
```

```
Out[6]: tensor([[ 0.6957, -1.8061, -1.1589,  0.3255, -0.6315]],  
          grad_fn=<EmbeddingBackward0>)
```

1) Index of training example [2]

2) Embedding matrix

0.3374	-0.1778	-0.3035	-0.5880	1.5810
1.3010	1.2753	-0.2010	-0.1606	-0.4015
0.6957	-1.8061	-1.1589	0.3255	-0.6315
-2.8400	-0.7849	-1.4096	-0.4076	0.7953

3) Look up entries by index

0.6957	-1.8061	-1.1589	0.3255	-0.6315
--------	---------	---------	--------	---------

[2]



扩展--torch.nn.Embedding(num_idx, out_dim)

In [7]: `idx = torch.tensor([2, 3, 1])`
`embedding(idx)`

Out[7]: `tensor([[ 0.6957, -1.8061, -1.1589, 0.3255, -0.6315],`
 `[-2.8400, -0.7849, -1.4096, -0.4076, 0.7953],`
 `[ 1.3010, 1.2753, -0.2010, -0.1606, -0.4015]],`
 `grad_fn=<EmbeddingBackward0>)`

1) Indices of 3 training examples

$$\begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix}$$

2) Embedding matrix

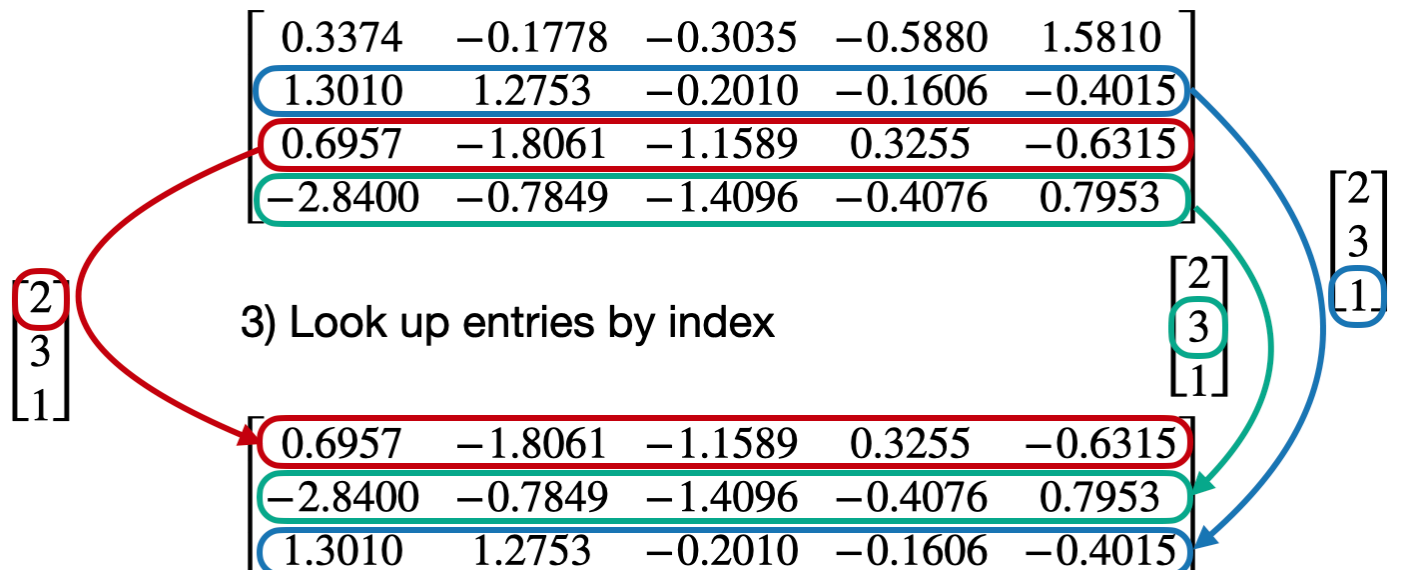
0.3374	-0.1778	-0.3035	-0.5880	1.5810
1.3010	1.2753	-0.2010	-0.1606	-0.4015
0.6957	-1.8061	-1.1589	0.3255	-0.6315
-2.8400	-0.7849	-1.4096	-0.4076	0.7953

3) Look up entries by index

$$\begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix}$$

0.6957	-1.8061	-1.1589	0.3255	-0.6315
-2.8400	-0.7849	-1.4096	-0.4076	0.7953
1.3010	1.2753	-0.2010	-0.1606	-0.4015

$$\begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix}$$


2.7 创建 token 向量

- 通过一个实际示例来了解 token ID 到嵌入向量转换的工作原理
 - 打印语句打印嵌入层的底层权重矩阵（形状为6x3）：



```
print(embedding_layer.weight)
```



```
Parameter containing:  
tensor([[ 0.3374, -0.1778, -0.1690],  
        [ 0.9178,  1.5810,  1.3010],  
        [ 1.2753, -0.2010, -0.1606],  
        [-0.4015,  0.9666, -1.1481],  
        [-1.1589,  0.3255, -0.6315],  
        [-2.8400, -0.7849, -1.4096]], requires_grad=True)
```

2.7 创建 token 向量

- 通过一个实际示例来了解 token ID 到嵌入向量转换的工作原理
 - 将 token ID 为 3 的单词转换为词向量：

```
[ ] print(embedding_layer(torch.tensor([3])))
```

```
⇒ tensor([[ -0.4015,  0.9666, -1.1481]], grad_fn=<EmbeddingBackward0>)
```

```
0 [[ 0.3374, -0.1778, -0.1690],  
1 [ 0.9178,  1.5810,  1.3010],  
2 [ 1.2753, -0.2010, -0.1606],  
3 [-0.4015,  0.9666, -1.1481],  
4 [-1.1589,  0.3255, -0.6315],  
5 [-2.8400, -0.7849, -1.4096]]
```

2.7 创建 token 向量

- 通过一个实际示例来了解 token ID 到嵌入向量转换的工作原理
 - 将 `input_ids` 全部转换为词向量:



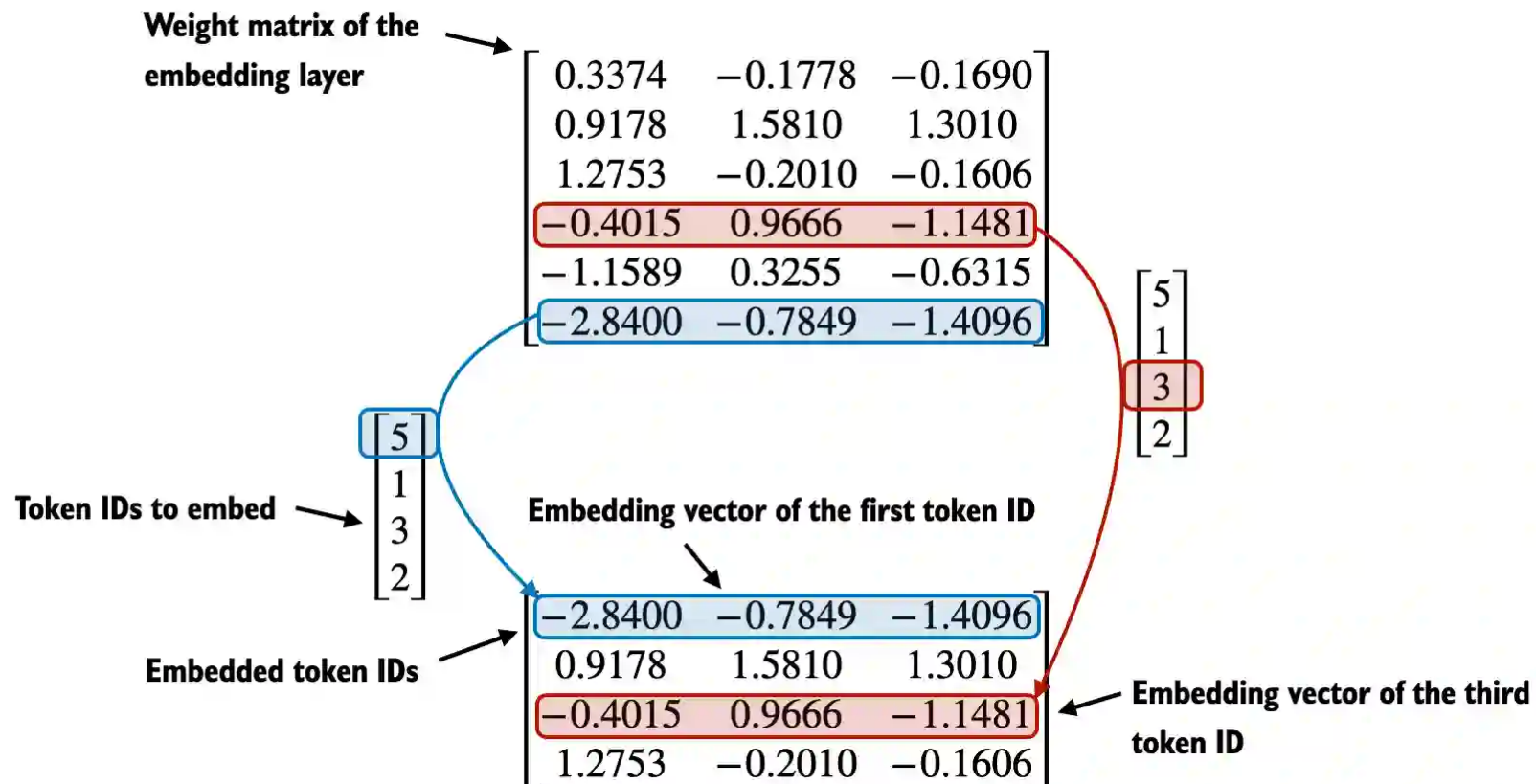
```
print(embedding_layer(input_ids))
```



```
tensor([[ 1.2753, -0.2010, -0.1606],  
        [-0.4015,  0.9666, -1.1481],  
        [-2.8400, -0.7849, -1.4096],  
        [ 0.9178,  1.5810,  1.3010]], grad_fn=<EmbeddingBackward0>)
```

2.7 创建 token 向量

- 嵌入层本质上是一种查找操作，它通过标记 ID 从嵌入层的权重矩阵中检索行



练习： Embedding Layer 实践

- 在本题中，我们将使用 `tokenizer` 对 `text = “我正在学习Natural Language Processing，非常有意思！”` 进行分词，并使用 PyTorch 的 `Embedding Layer` 获取词向量表示。
- 提示： `vocab_size = tokenizer.n_vocab`

2.8 编码单词位置

嵌入层将相同的token ID 转换为相同的向量表示，无论它位于输入序列中的哪个位置。例如，token ID 2，无论它位于 token ID 输入向量中的第一个还是第四个位置，都会产生相同的嵌入向量

Weight matrix of the embedding layer

$$\begin{bmatrix} 0.3374 & -0.1778 & -0.1690 \\ 0.9178 & 1.5810 & 1.3010 \\ 1.2753 & -0.2010 & -0.1606 \\ -0.4015 & 0.9666 & -1.1481 \\ -1.1589 & 0.3255 & -0.6315 \\ -2.8400 & -0.7849 & -1.4096 \end{bmatrix}$$

Token IDs to embed

fox
jumps
over
fox

$$\begin{bmatrix} 2 \\ 3 \\ 5 \\ 2 \end{bmatrix}$$

The same token IDs result in the same embedding vectors

$$\begin{bmatrix} 1.2753 & -0.2010 & -0.1606 \\ -0.4015 & 0.9666 & -1.1481 \\ -2.8400 & -0.7849 & -1.4096 \\ 1.2753 & -0.2010 & -0.1606 \end{bmatrix}$$

$$\begin{bmatrix} 2 \\ 3 \\ 5 \\ 2 \end{bmatrix}$$

fox
jumps
over
fox

2.8编码单词位置

- 当输入是batch_size为2，窗口大小为4时，输入大小为[2,4]

`nn.Embedding(10, 3)`

输入x:[2, 4]

1	2	4	5
4	3	2	9

X_0	1.4212	0.6127	-1.1126
X_1	0.4294	-1.0121	-1.8348
X_2	-0.0315	-1.2234	-0.4589
X_3	0.6131	-0.4381	0.1253
X_4	-1.0621	-0.1466	1.7412
X_5	1.0708	-0.7888	-0.0177
X_6	-0.5979	0.6465	0.6508
X_7	-0.5608	-0.3802	-0.4206
X_8	1.1516	0.4091	1.2477
X_9	-0.5753	0.1394	2.3447

输出y:[2, 4, 3]

X_1	X_2	X_4	X_5
X_4	X_3	X_2	X_9

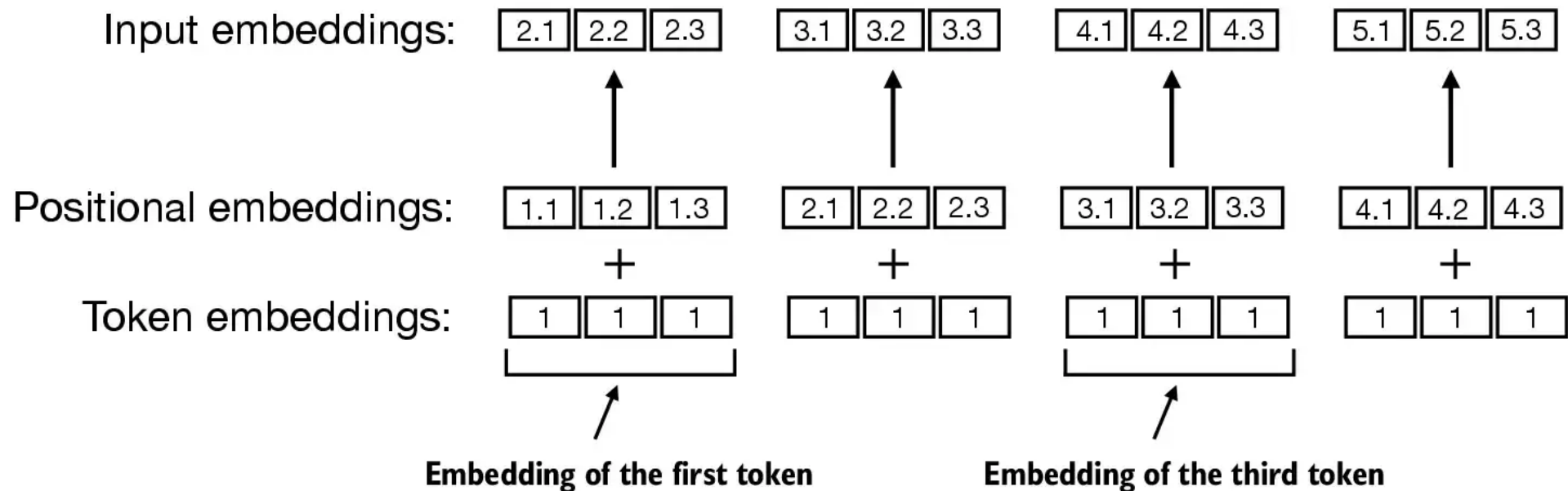
X_1	0.4294	-1.0121	-1.8348
X_2	-0.0315	-1.2234	-0.4589
X_4	-1.0621	-0.1466	1.7412
X_5	1.0708	-0.7888	-0.0177

X_4	-1.0621	-0.1466	1.7412
X_3	0.6131	-0.4381	0.1253
X_2	-0.0315	-1.2234	-0.4589
X_9	-0.5753	0.1394	2.3447

2.8 编码单词位置

- 位置感知

- 相对位置embeddings: token之间的相对位置和距离
- 绝对位置embeddings: 对于输入序列中的每个位置，都会将唯一的embeddings添加到token的词向量中以传达其确切位置。

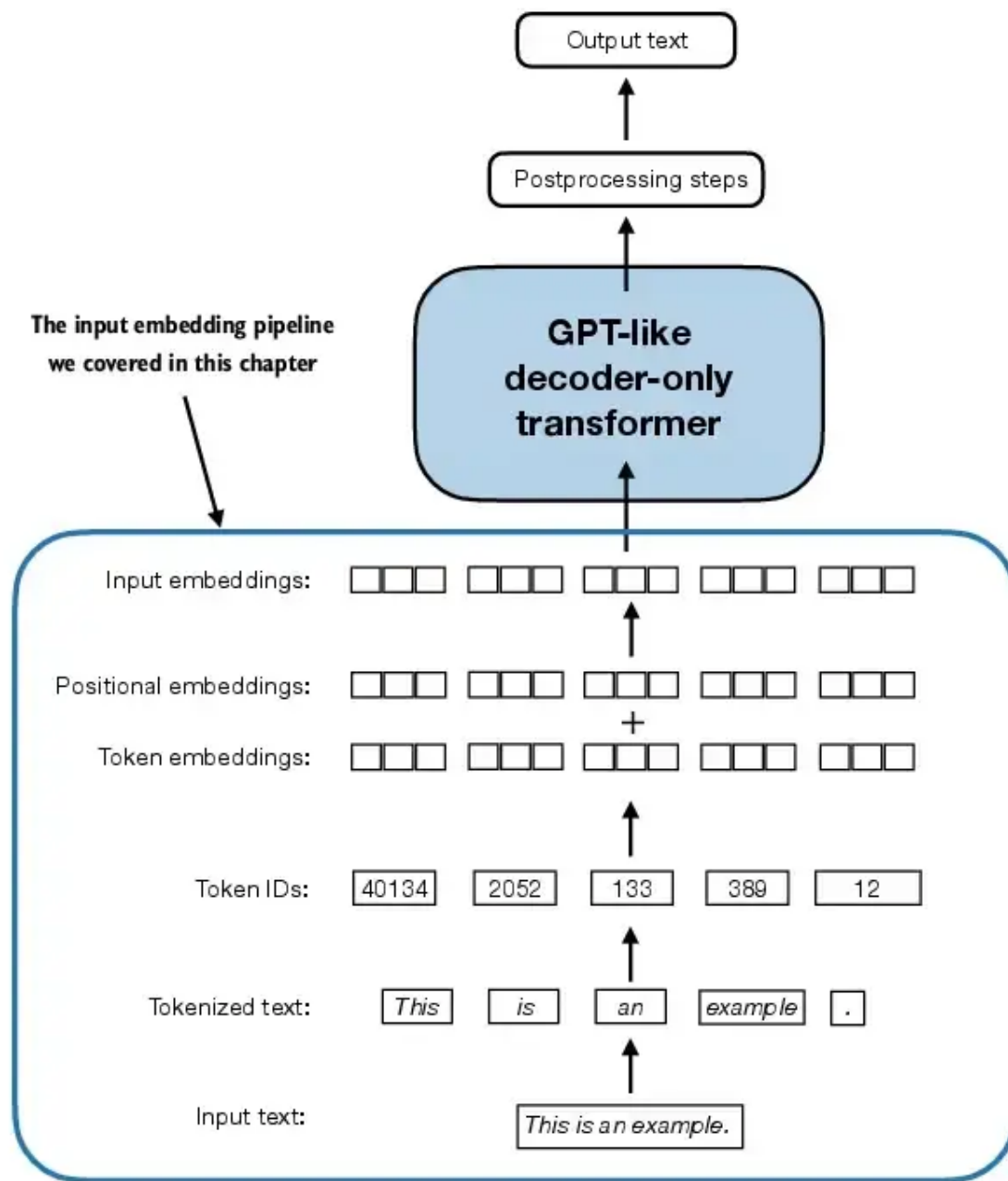


位置向量具有与原始标记嵌入相同的维度。为简单起见，标记嵌入显示为值 1

2.8编码单词位置

- **位置感知embeddings**
 - **相对位置embeddings**: token之间的相对位置和距离
 - **绝对位置embeddings**: 对于输入序列中的每个位置，都会将唯一的embeddings添加到token的词向量中以传达其确切位置。
 - **OpenAI 的 GPT 模型使用绝对位置embeddings**，这些embeddings在训练过程中进行了优化

本章总结



谢谢！ 欢迎提问！

Instructor: Xiaohan Ma(马晓寒), Ph.D.

Graduate School of Information and Communication Technology,

Ajou University, Korea

maxiaohan@ajou.ac.kr